Supplement to: *Intermediate abundance promotes speciation when dispersal is limited*

S1 Computation set-up

We created a make use of a custom R package *abundolism* which can be installed from github.

```
devtools::install_github("ajrominger/abundolism")
```

The following R computing environment is used for all simulations and analyses:

```
library(abundolism)
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(ggplot2)
library(ggpointdensity)
library(cowplot)
library(knitr)

sessionInfo() |>
    print(locale = FALSE)
```

```
R version 4.5.2 (2025-10-31)
Platform: aarch64-apple-darwin20
Running under: macOS Sequoia 15.0.1

Matrix products: default
BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framewo
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

other attached packages:
[1] knitr_1.50           cowplot_1.2.0.9000   ggpointdensity_0.2.1
[4] ggplot2_4.0.1        dplyr_1.1.4          abundolism_0.1.0

loaded via a namespace (and not attached):
 [1] vctrs_0.6.5        cli_3.6.5          rlang_1.1.6        xfun_0.52
 [5] generics_0.1.4     S7_0.2.1           jsonlite_2.0.0     glue_1.8.0
 [9] htmltools_0.5.8.1  scales_1.4.0       rmarkdown_2.29     grid_4.5.2
[13] evaluate_1.0.4     tibble_3.3.0       MASS_7.3-65        fastmap_1.2.0
[17] yaml_2.3.10        lifecycle_1.0.4    compiler_4.5.2     RColorBrewer_1.1-3
[21] Rcpp_1.1.0         pkgconfig_2.0.3    rstudioapi_0.17.1  farver_2.1.2
[25] digest_0.6.39      R6_2.6.1           tidyselect_1.2.1   pillar_1.11.0
[29] magrittr_2.0.4     withr_3.0.2        tools_4.5.2        gtable_0.3.6
```

The quarto (Allaire et al. 2025) document used to generate this supplement can be found with the R command `system.file("ab_supp.qmd", package = "abundolism")`.

S2 Simulation details

The simulation tracks

- all local population sizes
- running mean population size per population, per iteration
- time relative to all rates (i.e. not simply number of iterations)
- whether an incipient speciation event has occurred
- the wait time to full speciation
- whether full speciation has occurred

Each iteration an event is chosen at random. Possible events for each local population are

- local birth

- local death
- local speciation
- local immigration

There is also one possible global event: immigration from the source pool.

If there are n_p local populations then the total number of possible events are $n_p \times 4 + 1$.

Given that the size of a local population is written x_i, the probability of each event is proportional to its respective rate:

$$\mathbb{P}(\text{birth in pop. } i) = \frac{\lambda x_i}{R} \tag{1}$$

$$\mathbb{P}(\text{death in pop. } i) = \frac{\mu x_i}{R} \tag{2}$$

$$\mathbb{P}(\text{speciation in pop. } i) = \frac{\nu x_i}{R} \tag{3}$$

$$\mathbb{P}(\text{local immigration from pop. } i) = \frac{m_{prop}\gamma x_i}{R} \tag{4}$$

$$\mathbb{P}(\text{global immigration}) = \frac{\gamma}{R} \tag{5}$$

where $R = \gamma + \left(\sum_i x_i\right)(\lambda + \mu + \nu + m_{prop}\gamma)$ is a normalization constant.

In the case of immigration (both local and from the source pool) the recipient of the dispersing individual must also be chosen. We choose the recipient local population at random with equal probability across local populations.

Once an event is randomly sampled, the variables being tracked are updated. Birth and immigration (local or global) augment a local population by 1. Death decrements a local population by 1. Immigration events have no impact on the source or the dispersing individual. Time is updated following the Gillespie algorithm: the sojourn time between events is a random variable with an exponential probability distribution with rate R: $s \sim Exp(R)$. Each time step a random sojourn time is sampled and adding to the running total time since the start of the simulation.

If a population experiences local extirpation all of its speciation-related information is re-set including removing any incipient speciation and restoring the wait time to full speciation back to the default of τ.

If full speciation is achieved the simulation stops. If no speciation occurs but the specified number of simulation iterations is reached, the simulation stops. Once stopped, the simulation returns the tracked variables of time, mean population size, and whether full speciation has occurred.

S3 Simulation experiment details

Simulation parameters are drawn from uniform distributions with the following minimum and maximum limits:

```
par_range <- data.frame(
    param = c("la", "mu", "g", "m_prop", "nu", "tau", "xi"),
    min   = c(0.1,  0.1,  0.1,  0,        0,     20,  10),
    max   = c(4,    4,    1,    0.1,      0.01,  200, 100)
)


# fixed params
np <- 2
nstep <- 10000

# number of simulation replicates
nrep <- 5000

# format param names for printing
mutate(par_range,
       param = sprintf("`%s`", param)) |>
    kable(format.args = list(scientific = FALSE))
```

param	min	max
`la`	0.1	4.00
`mu`	0.1	4.00
`g`	0.1	1.00
`m_prop`	0.0	0.10
`nu`	0.0	0.01
`tau`	20.0	200.00
`xi`	10.0	100.00

The parameters governing number of populations (`np`) and number of time steps (`nstep`) do not vary across simulations and are set to `np = 2` and `nstep = 10^4`. We simulate a total of `nrep = 5000` replicates, each with a unique, randomly sampled set of parameter values.

S3.1 Running the simulation experiment

Parameter values for each simulation replicate are stored in a `data.frame` with one column for each parameter.

```r
# change or remove for different results
# set.seed(123)

# data.frame of randomly sampled param values
# one param per column
pars <- sapply(par_range$param, function(p) {
    runif(nrep,
          par_range[par_range$param == p, "min"],
          par_range[par_range$param == p, "max"])
}) |>
    as.data.frame()

# pars$tau <- 10 / (pars$la + pars$mu + runif(nrep, 0, 1))
```

```r
# now run the simulation, see `?sim_BDI_spec` for details
# on input and output
sim_dat <- sim_BDI_spec(la = pars$la, mu = pars$mu, g = pars$g,
                        m_prop = pars$m_prop, nu = pars$nu,
                        tau = pars$tau, xi = pars$xi,
                        np = np, nstep = nstep)
```

Now we can plot the results and find out how abundance relates to speciation in this model

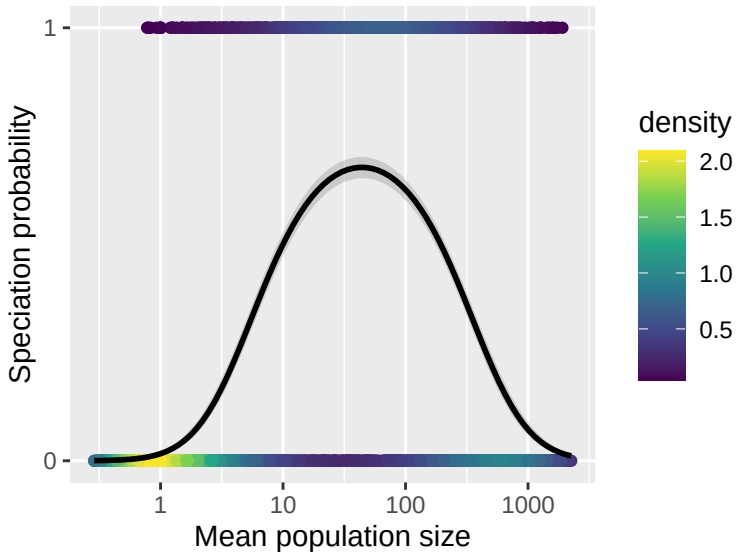
```r
ggplot(sim_dat,
       aes(x = tau, speciation)) +
    geom_pointdensity(method = "kde2d") +
    scale_shape_binned() +
    scale_color_viridis_c() +
    # scale_x_log10() +
    geom_smooth(method = "glm",
                method.args = list(family = "binomial"),
                color = "black")

ggplot(sim_dat, aes(niter, time)) +
    geom_pointdensity(method = "kde2d") +
    scale_color_viridis_c() +
    # scale_x_log10() +
    scale_y_log10() +
    facet_wrap(vars(speciation))


ggplot(sim_dat, aes(time, mean_pop_size)) +
```

```r
    geom_point(data = select(sim_dat, !speciation),
               mapping = aes(time, mean_pop_size), color = "gray50") +
    geom_pointdensity() +
    facet_wrap(vars(speciation)) +
    scale_x_log10() +
    scale_y_log10() +
    scale_color_viridis_c(trans = "log10")

ggplot(sim_dat, aes(time, mean_pop_size)) +
    geom_point(data = select(sim_dat, !speciation),
               mapping = aes(time, mean_pop_size), color = "gray50") +
    geom_pointdensity() +
    facet_wrap(vars(speciation)) +
    scale_x_log10() +
    scale_y_log10() +
    scale_color_viridis_c(trans = "log10")

ggplot(sim_dat[sim_dat$time < 100, ], aes(time, mean_pop_size)) +
    geom_pointdensity() +
    facet_wrap(vars(speciation)) +
    scale_x_log10() +
    scale_y_log10() +
    scale_color_viridis_c(trans = "log10")

ggplot(sim_dat, aes(time)) +
    geom_histogram() +
    facet_wrap(vars(speciation), ncol = 1) +
    scale_x_log10()
```

This is the main figure

```r
sim_fig <- ggplot(sim_dat,
                  aes(x = mean_pop_size, y = speciation)) +
    geom_pointdensity(method = "kde2d") +
    scale_color_viridis_c() +
    # scale_size_continuous(range = c(0.001, 4)) +
    scale_x_log10() +
    scale_y_continuous(breaks = c(0, 1)) +
    xlab("Mean population size") +
    ylab("Speciation probability") +
    theme(panel.grid.minor.y = element_blank()) +
    geom_smooth(method = glm, formula = y ~ x + I(x^2),
```

```
                 method.args = list(family = "binomial"), color = "black")

sim_fig
```



S4 Analyzing abundance and richness of arthropod genera in Hawai‘i

```
# consider making exported data and sourcing from package
# read gruner data
arth <- read.csv("raw_data/arth.csv")

# read checklist data
arth_check <- read.csv("raw_data/hawaii_arthropod_checklist.csv")

# calculate richness of genera from checklist
gen_rich <- filter(arth_check, Genus != "") |>
    group_by(Genus) |>
    summarize(nspp = length(unique(Genus.and.species))) |>
    rename(genus = Genus)

# calculate abundances of genera from survey data
gen_abund <- group_by(arth, genus, species_code) |>
    summarize(abund = n()) |>
```

```r
    ungroup() |>
    group_by(genus) |>
    summarize(hmean_abund = n() / sum(1 / abund),
              mean_abund = mean(abund))
```

```
`summarise()` has grouped output by 'genus'. You can override using the
`.groups` argument.
```

```r
# `gen` holds data ready to analyze for richness ~ abundance
gen <- inner_join(gen_rich, gen_abund)
```

```
Joining with `by = join_by(genus)`
```

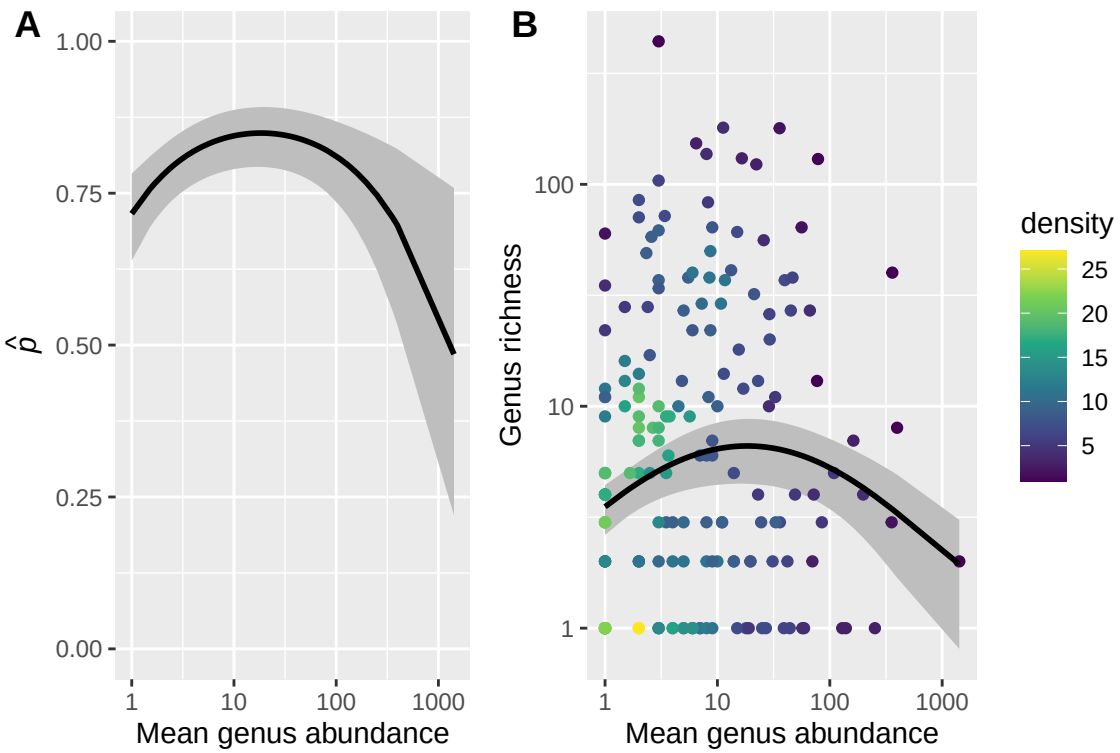
```r
# add column for log abundance
gen$log_abund <- log(gen$mean_abund)

# quadratic model
arth_mod <- glm_bgeom(nspp ~ log_abund + I(log_abund^2), data = gen)

# straight line model
arth_mod0 <- glm_bgeom(nspp ~ log_abund, data = gen)
```

```r
arth_lrt <- lrt_bgeom(arth_mod0, arth_mod)
arth_lrt
```

```
Likelihood ratio test

Model 0: nspp ~ log_abund
Model 1: nspp ~ log_abund + I(log_abund^2)

        #Df  LogLik Df  Chisq Pr(>Chisq)
Model 0   3 -675.84
Model 1   4 -670.31  1 11.049  0.0008874 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
arth_mod_ci <- cbind(mean_abund = gen$mean_abund,
                     ci_bgeom(arth_mod, type = "response"))
```

```r
arth_p_ci <- cbind(mean_abund = gen$mean_abund,
                   ci_bgeom(arth_mod, type = "prob"))

gp_nspp <- ggplot(arth_mod_ci, aes(mean_abund, ymin = y_lwr, ymax = y_upr)) +
    geom_ribbon(fill = "gray") +
    scale_x_log10() +
    scale_y_log10() +
    xlab("Mean genus abundance") +
    ylab("Genus richness") +
    geom_pointdensity(data = gen, mapping = aes(mean_abund, nspp),
                      inherit.aes = FALSE) +
    scale_color_viridis_c() +
    geom_line(data = arth_mod_ci,
              mapping = aes(mean_abund, y_hat),
              linewidth = 1)


gp_prb <- ggplot(arth_p_ci, aes(mean_abund, y = p_hat, ymin = p_lwr, ymax = p_upr)) +
    geom_ribbon(fill = "gray") +
    geom_line(linewidth = 1) +
    scale_x_log10() +
    ylim(0, 1) +
    xlab("Mean genus abundance") +
    ylab(expression(italic(hat(p))))

gp_arth <- plot_grid(gp_prb, gp_nspp, rel_widths = c(0.75, 1),
                     labels = c("A", "B"), label_x = 0.02)
gp_arth
```



Supplementary Figure S1

```
gen <- mutate(
    gen,
    yhat = pred_geom(arth_mod,
                     newdata = data.frame(log_abund = log_abund)))

ggplot(gen, aes(mean_abund, nspp)) +
    geom_pointdensity(method = "kde2d") +
    scale_x_continuous(trans = "log10") +
    scale_y_continuous(trans = "log10") +
    scale_color_viridis_c() +
    geom_line(aes(mean_abund, yhat))




# outliers
# gen[gen$nspp > 300, ]

mean(gen$nspp[gen$mean_abund <= 3])
```

```r
?filter


x <- 1:10
stats::filter(x, rep(1, 3), sides = 1) / 3
```

Double check it's not just outliers

```r
arth_mod <- glm_bgeom(nspp ~ log_abund + I(log_abund^2),
                      data = gen[gen$nspp < 400 & gen$mean_abund < 1000, ])
arth_mod0 <- glm_bgeom(nspp ~ log_abund, data = gen[gen$nspp < 400 & gen$mean_abund < 1000, ]



lrt_bgeom(arth_mod0, arth_mod)
```

```
Likelihood ratio test

Model 0: nspp ~ log_abund
Model 1: nspp ~ log_abund + I(log_abund^2)

        #Df  LogLik Df  Chisq Pr(>Chisq)
Model 0   3 -662.62
Model 1   4 -658.17  1 8.9088   0.002838 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

References

Allaire, J. J., Teague, C., Scheidegger, C., Xie, Y., Dervieux, C. and Woodhull, G. 2025. Quarto.